From C to C++ III

Concurrency

Roberto Masocco
roberto.masocco@uniroma2.it

Tor Vergata University of Rome
Department of Civil Engineering and Computer Science Engineering

April 30, 2026

Roadmap

1. std::chrono
2. std::thread
3. Race conditions
4. Mutexes and locks
5. Condition variables
6. Atomic operations

Roadmap

This lecture is here.

Code examples for this lecture are in `tools/cpp-examples/3/`.

Roadmap

1 std::chrono

2 std::thread

3 Race conditions

4 Mutexes and locks

5 Condition variables

6 Atomic operations

std::chrono

Time in C vs C++

In C, time handling is primitive: `time()`, `gettimeofday()`, `clock_gettime()` all return integers or structs with arbitrary units, and arithmetic on them is manual and error-prone.

C++ provides `std::chrono` — a **type-safe** time library in which **durations**, **time points**, and **clocks** are distinct types.
The compiler prevents mixing incompatible units at **compile time**:

- you cannot accidentally add seconds to a raw integer;
- **conversions** between units (e.g., milliseconds to seconds) are explicit or lossless;
- the units are encoded **in the type itself**.

`std::chrono` is used throughout the C++ standard library and in ROS 2 (**timers**, **rates**, **timeouts**).

std::chrono

Durations and literals

An `std::chrono::duration` represents a **time span**.

The standard library provides ready-made aliases for common units:

`std::chrono::nanoseconds`, `microseconds`, `milliseconds`, `seconds`, `minutes`, `hours`.

C++14 introduced **chrono literals**, which allow writing durations in a natural syntax.

```
#include <chrono>
using namespace std::chrono_literals;

auto half_second = 500ms;        // std::chrono::milliseconds
auto two_seconds = 2s;           // std::chrono::seconds
auto total       = half_second + two_seconds; // 2500ms
std::chrono::milliseconds period(100);

```

Listing 1: Chrono literals and duration arithmetic.

std::chrono

Clocks and time points

A **clock** provides access to the current time. The most important ones are:

- `std::chrono::steady_clock`: monotonic, never adjusted — use it for **measuring elapsed time**;
- `std::chrono::system_clock`: wall-clock time, may be adjusted by NTP — use it for **timestamps**.

A `time_point` represents a specific instant, obtained from a clock.
Subtracting two time points yields a **duration**.

std::chrono

Clocks and time points

```
using clk = std::chrono::steady_clock;

auto start = clk::now();
// ... do some work ...
auto end = clk::now();

auto elapsed = std::chrono::duration_cast<
  std::chrono::milliseconds>(end - start);
std::cout << "Elapsed: " << elapsed.count() << " ms\n";

```

Listing 2: Measuring elapsed time with `steady_clock`.

`std::this_thread::sleep_for()` suspends the calling thread for a given duration. It is the C++ replacement for `usleep()` and `nanosleep()`, and it accepts any `std::chrono::duration`.

```
#include <chrono>
#include <thread>

using namespace std::chrono_literals;

std::this_thread::sleep_for(200ms);  // sleep 200 ms
std::this_thread::sleep_for(1s);     // sleep 1 second

```

Listing 3: Sleeping with `chrono` durations.

Roadmap

1 std::chrono

2 std::thread

3 Race conditions

4 Mutexes and locks

5 Condition variables

6 Atomic operations

std::thread

From POSIX to C++

You already know threads from your OS course: a thread is an **independent flow of execution** within a process, sharing its **address space** and **resources**.

In C/POSIX, threads are managed through the `pthread` API: `pthread_create`, `pthread_join`, `pthread_detach` — functional but verbose.

C++11 introduced `std::thread`: a **portable**, **RAII-aware wrapper** around the platform's threading primitives.
The underlying model is identical:

- threads share the process memory space;
- each thread has its own stack;
- scheduling is managed by the operating system kernel.

Same model, cleaner interface, fewer mistakes.

std::thread

Creating threads

An `std::thread` is constructed by passing any **callable** and its arguments.
The thread starts executing **immediately** upon construction, arguments are **copied** by default.

```
#include <iostream>
#include <thread>

void greet(const std::string & name) {
  std::cout << "Hello from " << name << "\n";
}

std::thread t1(greet, "thread_1");

std::thread t2([]() {std::cout << "Hello from lambda\n";});

```

Listing 4: Creating threads from a function and a lambda.

std::thread

join() and detach()

Every `std::thread` must be either **joined** or **detached** before it is destroyed.

`t.join()` blocks the calling thread until `t` finishes — the standard way to **wait for completion**.

`t.detach()` releases ownership: the thread runs independently and its resources are reclaimed by the OS when it finishes — use sparingly and only for **fire-and-forget** tasks.

```
std::thread t1(greet, "worker_1");
std::thread t2(greet, "worker_2");

t1.detach(); // do not wait for t1 to finish
t2.join();   // wait for t2 to finish

```

Listing 5: Joining threads.

Roadmap

1 std::chrono

2 std::thread

3 Race conditions

4 Mutexes and locks

5 Condition variables

6 Atomic operations

Race conditions

Shared mutable state

When two or more threads access the **same memory location** and at least one of them **writes** to it, the result depends on the **relative order of execution** — which is determined by the OS scheduler (or, even, the CPU) and is **non-deterministic**.

Definition of race condition

A **race condition** occurs when the correctness of a program depends on the specific **interleaving** of operations from concurrent threads on shared data.

You have studied this in your OS course. In C++ the problem is identical — the threading model is the same — but the solutions are expressed differently.

Race conditions

Why a simple increment breaks

Consider two threads each incrementing a shared `int` one million times.

```
int counter = 0;
void increment(int n) {
  for (int i = 0; i < n; ++i) counter++;
}

int main() {
  std::thread t1(increment, 1000000);
  std::thread t2(increment, 1000000);
  t1.join();
  t2.join();
  // counter == 2000000?
}

```

Listing 6: A racy increment — result will be wrong.

Race conditions

Anatomy of the race

The expression `counter++` is **not a single operation**. It decomposes into three steps:

1. **read** the current value of `counter` from memory into a register;
2. **modify** (increment) the value in the register;
3. **write** the new value back to memory.

If two threads interleave in the middle:

1. Thread A reads `counter = 5`
2. Thread B reads `counter = 5` (same value!)
3. Thread A writes `counter = 6`
4. Thread B writes `counter = 6` — **one increment is lost**.

Non-determinism

Running the same program twice may produce **different results**. These bugs are among the **hardest to find and reproduce**.

Race conditions

Solutions

The fundamental approaches to eliminating data races are

- **mutual exclusion**: ensure that only one thread accesses the shared data at a time — using **mutexes** and **locks**;
- **atomic operations**: use hardware-level instructions that perform read-modify-write as a **single, indivisible step**;
- **avoiding shared state**: design the system so that threads do not share mutable data at all (message-passing, thread-local storage).

We will now see the C++ standard library tools for the first two approaches — they correspond directly to the POSIX primitives you already know, wrapped in RAII.

Roadmap

1 std::chrono

2 std::thread

3 Race conditions

4 Mutexes and locks

5 Condition variables

6 Atomic operations

Mutexes and locks

std::mutex

std::mutex is the C++ equivalent of pthread_mutex_t. It provides two operations:

- lock(): **acquires** the mutex; if another thread holds it, **blocks** until it is released;
- unlock(): **releases** the mutex.

```
int counter = 0;
std::mutex mtx;
void increment(int n) {
  for (int i = 0; i < n; ++i) {
    mtx.lock();
    counter++;
    mtx.unlock();
  }
}

```

Listing 7: Protecting a shared counter with std::mutex.

Mutexes and locks

std::lock_guard

std::lock_guard<std::mutex> applies RAII to mutex locking.

```
void increment(int n) {
  for (int i = 0; i < n; ++i) {
    std::lock_guard<std::mutex> lock(mtx);
    counter++;
  } // lock destroyed here: mutex unlocked
}

```

Listing 8: RAII locking with lock_guard.

RAII closing the loop from Lecture 1: the resource is a mutex lock, the object is the guard. Never call lock()/unlock() manually — **always use a guard**.

Mutexes and locks

std::unique_lock

std::unique_lock<std::mutex> is a more flexible alternative to lock_guard:

- it can be **deferred** (constructed without locking), **manually locked/unlocked**, and **moved** between scopes;
- it is **required** by std::condition_variable.

```
std::mutex mtx;

std::unique_lock<std::mutex> lock(mtx, std::defer_lock);
// mutex NOT locked yet

lock.lock();      // lock manually when needed
// ... critical section ...
lock.unlock();    // unlock manually before destruction

```

Listing 9: unique_lock with deferred locking.

Roadmap

1 std::chrono

2 std::thread

3 Race conditions

4 Mutexes and locks

5 Condition variables

6 Atomic operations

Condition variables

The waiting problem

Sometimes a thread must wait for a specific **condition** to become true before it can proceed — for example:

- a consumer thread waiting until a **queue is non-empty**;
- a worker waiting until a **task is available**;
- a thread waiting for a **configuration to be loaded**.

Polling the condition in a busy loop (**spin-waiting**) wastes CPU cycles.
Sleeping for a fixed time risks missing the event or adding unnecessary latency.

Definition of condition variable

A **condition variable** is a synchronization primitive that allows a thread to **sleep efficiently** until another thread **signals** that a condition of interest has changed.

Condition variables

std::condition_variable

`std::condition_variable` is the C++ wrapper around `pthread_cond_t`.
It works together with a `std::mutex` and a `std::unique_lock`:

- `wait(lock)`: atomically releases the mutex and puts the thread to sleep; when awoken, it re-acquires the mutex before returning;
- `notify_one()`: wakes up one waiting thread;
- `notify_all()`: wakes up all waiting threads.

Condition variables

```
std::mutex mtx;
std::condition_variable cv;
bool ready = false;

// WAITING THREAD
std::unique_lock<std::mutex> lock(mtx);
cv.wait(lock, [&]{ return ready; }); // captures all by ref.

// SIGNALING THREAD
{
  std::lock_guard<std::mutex> lock(mtx);
  ready = true;
}
cv.notify_one();

```

Listing 10: Basic condition variable pattern.

Condition variables

Spurious wakeups

A thread waiting on a condition variable may be **woken up without a corresponding notification** — this is a **spurious wakeup**, an artifact of the underlying implementation.

For this reason, the condition must **always be rechecked after waking up**.
The overload of `wait()` that accepts a **predicate** (a callable returning `bool`) handles this automatically:

- `cv.wait(lock, predicate)` is equivalent to:
  ```
  while (!predicate()) cv.wait(lock);
  ```

Always use a predicate

Calling `wait(lock)` without a predicate is a common source of subtle bugs: the thread wakes up, assumes the condition is true, and proceeds — but it may not be. **Always pass a predicate.**

Condition variables

`notify_one()` vs `notify_all()`

`notify_one()` wakes up **one** waiting thread (chosen by the OS).
Use it when a single event satisfies a single waiter — e.g., one item added to a queue, one consumer to wake.

`notify_all()` wakes up **all** waiting threads.
Use it when the condition change may be relevant to multiple waiters — e.g., a shutdown signal that all workers must see.

After waking, each thread re-acquires the mutex and re-checks the predicate.
If the condition is no longer true (another thread consumed the event first), the thread goes back to sleep automatically — the predicate loop handles this.

When in doubt, prefer `notify_all()` — it is safe,
and the performance difference is negligible for most applications.

Roadmap

1 std::chrono

2 std::thread

3 Race conditions

4 Mutexes and locks

5 Condition variables

6 Atomic operations

Atomic operations

An **atomic operation** is one that completes as an **indivisible step** from the perspective of all other threads — no other thread can observe a partial result.

Modern processors support atomic **read-modify-write** instructions at the hardware level (*e.g.*, `lock xadd` on x86, `ldxr/stxr` on ARM).
C++ exposes them through `std::atomic<T>`.

Atomics are appropriate when

- the shared state is a **single variable** of a simple type (`int`, `bool`, pointer);
- the operation on it is **self-contained** (increment, compare-and-swap, flag set/clear);
- you need a **memory barrier** enforcing an **order**:
  - define **who wins a race**;
  - ensure that **all cache-to-memory writes are visible to everyone** from the next line.

For anything more complex than a single variable, use a **mutex**.

Atomic operations

`std::atomic<T>`

`std::atomic<T>` wraps a value of type `T` and guarantees that all operations on it are atomic. The most common operations are:

- `store(value)`: atomically write;
- `load()`: atomically read;
- `fetch_add(n)`: atomically read, add, and write; returns the old value;
- `compare_exchange()`: atomically read, compare, and overwrite with a different value if the comparison succeeds;
- the usual arithmetic operators (++, +=, etc.) are overloaded and atomic.

Atomic operations

std::atomic<T>

```
std::atomic<int> counter{0};
void increment(int n) {
  for (int i = 0; i < n; ++i)
    counter++; // atomic read-modify-write
}

int main() {
  std::thread t1(increment, 1000000);
  std::thread t2(increment, 1000000);
  t1.join();
  t2.join();
  std::cout << counter << std::endl; // Exactly 2000000
}
```

Listing 11: Fixing the counter race with `std::atomic`.

Atomic operations

Atomic flags

A very common use of atomics is a **stop flag**: a boolean that one thread sets to signal others to terminate their loops.

```
std::atomic<bool> running{true};
void worker() {
  while (running.load()) { /* work */ }
}

int main() {
  std::thread t(worker);
  /* ... */
  running.store(false); // Signal the worker to stop
  t.join();
}

```

Listing 12: Using an atomic flag to signal thread termination.

Atomic operations

Atomics vs mutexes

Atomics:

- protect a **single variable**;
- no waiting, no context switch — **lock-free**;
- faster for simple operations (counters, flags);
- acting as **memory barriers**.

Mutexes:

- protect an **arbitrary critical section** involving multiple variables and operations;
- a thread that cannot acquire the lock is put to sleep by the OS — **blocking**;
- the only correct choice when **a single variable is not enough**.